PowerShell — Complete Study Notes

DevOps Training | Cognizant Azure Environment | Comprehensive Reference

PHASE 1 — The Mental Model

1.1 — What is PowerShell and Why Does It Exist?

PowerShell is a task automation and configuration management framework built by Microsoft. It is both a command-line shell and a scripting language. Unlike the old Windows CMD shell, which passes plain text between commands, PowerShell passes objects. This is the single most important difference — everything in PowerShell is an object with properties and methods, not raw text strings.

Think of CMD as a walkie-talkie — you can send and receive words, but that is all. PowerShell is like a structured form — every piece of information is labelled, typed, and usable. When a command returns a list of services, it does not return a wall of text; it returns a collection of service objects, each carrying properties like Name, Status, and DisplayName that you can filter, sort, and act on directly.

PowerShell was designed for system administrators who need to automate repetitive tasks across Windows infrastructure. Instead of clicking through GUIs to start or stop services, check disk space, or configure users across hundreds of servers, an administrator writes a script once and runs it everywhere.

1.2 — The Verb-Noun Naming Convention

All standard PowerShell commands — called cmdlets (pronounced command-lets) — follow a Verb-Noun pattern. The verb describes what the command does, and the noun describes what it acts on. This convention makes PowerShell highly guessable: if you know you want to retrieve service information, you try Get-Service. If you want to stop it, you try Stop-Service.

- Get-Service — retrieves service information
- Stop-Service — stops a running service
- Start-Service — starts a stopped service
- Get-Process — retrieves running process information
- Get-ChildItem — lists files and folders
- Get-Date — returns the current date and time

⚠ NOTE: The verb-noun convention is not just a style guide — it is enforced. Microsoft publishes an approved list of verbs (Get, Set, New, Remove, Start, Stop, etc.). If you create a custom cmdlet and use a non-approved verb, PowerShell will show a warning.

1.3 — The Object Pipeline Philosophy

The pipeline is the | character between commands. It takes the output of one command and passes it as input to the next. In CMD and bash, pipelines pass text strings. In PowerShell, they pass full objects. This means you can filter on properties, sort by any column, and measure values — all without parsing text.

A real-world analogy: imagine a factory conveyor belt. Each item on the belt is a box with labelled compartments (Name, Status, Size). Each station on the belt can inspect the compartments, keep only the boxes it wants, rearrange them, or read specific compartments — without ever opening and repacking the boxes.

1.4 — PowerShell Console vs ISE vs Other Tools

Feature	Text Console	ISE	VS Code + Extension
IntelliSense	Tab completion only	Full IntelliSense	Full IntelliSense
Script editing	Basic	Built-in editor pane	Advanced editor
Debugging	Limited	Breakpoints and step-through	Full debugger
Remote connection	Supported	Supported	Supported
Run selection (F8)	No	Yes	Yes
Use case	Servers with no tools	Scripting and learning	Professional development

1.5 — Starting PowerShell Correctly

PowerShell can be started in several ways. The most important consideration is whether you need administrator (elevated) rights. Commands that interact with system-level components — like Get-Disk — will fail with a "permission denied" error unless PowerShell is running as administrator. This is a UAC (User Account Control) restriction introduced with Windows Vista.

- From Start menu — type powershell and press Enter
- As Administrator — right-click the shortcut and choose "Run as administrator"
- From CMD — type powershell to launch PowerShell inside a CMD window
- Permanent elevation — set "Run as administrator" in shortcut advanced properties

⚠ NOTE: When PowerShell is started from CMD, the window loses its characteristic blue colour. You are still in PowerShell, but the host environment is different. The $PSVersionTable variable will confirm which version is running.

1.6 — Checking the PowerShell Version

The $PSVersionTable variable is a built-in automatic variable that holds information about the currently running PowerShell version. Variables in PowerShell are always prefixed with the $ sign.

```
$PSVersionTable

# To force PowerShell to simulate an older version at launch:
powershell -version 2
```

`$PSVersionTable` → displays a table showing PSVersion, PSEdition, OS, and more

`-version 2` → starts PowerShell simulating version 2.0 (useful for compatibility testing)

⚠ NOTE: Simulating a lower version with -version is useful when writing scripts that need to run on older servers. Not all cmdlets exist in older versions.

PHASE 2 — Core Building Blocks

2.1 — Variables

A variable in PowerShell is a named memory location that holds a value. All variable names are prefixed with the dollar sign $ when you want to use the value. The actual name of the variable is the part without the $ sign — the $ is the instruction to PowerShell to look up and retrieve the value stored under that name.

Creating a variable is as simple as choosing a name and using the equality operator to assign a value. PowerShell is loosely typed by default, meaning it will guess the type of the value automatically. You can also declare the type explicitly, which is discussed in Section 2.4.

Creating and Using Variables

```
$CompName = "AzureVM01"
$portNumber = 8080
$CompName    # displays: AzureVM01
```

`$CompName = "AzureVM01"` → creates a variable named CompName and stores the text AzureVM01 in it

`"$CompName"` → double quotes → PowerShell replaces $CompName with its value (interpolation)

`'$CompName'` → single quotes → PowerShell treats it literally; outputs: $CompName (no substitution)

⚠ NOTE: Single quotes disable variable interpolation. Double quotes enable it. This distinction is critical when building filter strings, paths, or WQL queries where you need to mix literal text and variable values.

Variable Management Commands

Command	What it does
New-Variable -Name "x"	Creates variable with empty value (formal method)
Set-Variable -Name "x" -Value "y"	Sets variable value — useful when name is generated dynamically
Clear-Variable -Name "x"	Empties the value but keeps the variable in memory
Remove-Variable -Name "x"	Deletes the variable from memory entirely
Get-Command -Noun Variable	Lists all cmdlets related to variables

The most common everyday syntax is simply $name = value. The cmdlet forms (New-Variable, Set-Variable) are used when the variable name itself is generated dynamically at runtime, for example when variable names are constructed from a list.

⚠ NOTE: When passing the variable name to management cmdlets (New-Variable, Set-Variable, Clear-Variable, Remove-Variable), do NOT use the $ sign. You reference the name, not the value. $name means "give me the value", while name (without $) means "the variable named name".

The Variable: Virtual Drive

PowerShell exposes all session variables through a virtual drive called Variable:. You can browse it just like a file system using Get-ChildItem. This lets you list all variables in the current session, search for variables by mask, and confirm whether a variable exists.

```
Get-ChildItem Variable:          # list all session variables
Get-ChildItem Variable:comp*    # find variables whose name starts with comp
Remove-Item Variable:CompName   # delete variable via the drive
```

⚠ NOTE: Session scope matters: all user-defined variables are lost when the PowerShell session ends. Only PowerShell's own automatic variables (like $PSVersionTable, $_, $PSItem) are recreated automatically at the start of each new session.

Single Quotes vs Double Quotes — Key Distinction

Quote Type	Behaviour	Example Input	Actual Result
Double " "	Variable interpolation ON — $var replaced with its value	"Server: $CompName"	Server: AzureVM01

Single ' '	Variable interpolation OFF — text is literal	'Server: $CompName'	Server: $CompName

A common real-world error: you build a filter expression like "*$keyword*" and it works fine. Then you change the outer quotes to single quotes and suddenly nothing matches — because $keyword is no longer replaced with its value; PowerShell literally searches for the string "$keyword" in the data.

2.2 — List Variables (Arrays)

A list variable (array) holds multiple values under a single name. To create a list, separate values with commas. PowerShell treats a comma-separated list of values as an array automatically — this is different from a single string that contains commas as text.

```
$servers = "AzVM01", "AzVM02", "AzVM03"    # this is an array of 3 strings
$servers = "AzVM01, AzVM02, AzVM03"         # this is ONE string (not an array!)

# Read a list from a file (each line becomes one element)
$servers = Get-Content "C:\servers.txt"
```

`"val1", "val2", "val3"` → commas without outer quotes create an array

`"val1, val2, val3"` → single string containing commas — NOT an array

`Get-Content "file.txt"` → reads the file and returns each line as a separate element in a list

Accessing List Elements

```
$servers[0]          # first element (indexing starts at 0)
$servers[1]          # second element
$servers[-1]         # last element (negative index goes from the end)
$servers.Count       # number of elements
$servers.Length      # same as Count for arrays
$servers[0].Length   # length of the first element (as a string)
```

⚠ NOTE: PowerShell indexing starts at zero. $servers[0] is the first element. $servers[-1] is the last element — a very convenient shortcut. Accessing an out-of-range index does not throw an error; it returns empty.

Working with Lists in Practice

```
# Send list elements through the pipeline
$servers | ForEach-Object { Write-Host "Processing: $_" }

# Build paths dynamically per element
$servers | ForEach-Object { Get-Content "C:\logs\$_.txt" }

# Store command results as a list
$allServices = Get-Service
$runningServices = Get-Service | Where-Object { $_.Status -eq "Running" }
$stoppedCount = $allServices.Count - $runningServices.Count
```

2.3 — Variable Types

By default PowerShell uses dynamic (implicit) typing — it decides the type based on the value you assign. You can also force a specific type by putting the type name in square brackets before the variable name. This is called explicit type declaration and it uses the .NET type system.

```
[int]$port = 443                # integer only
[string]$serverName = "AzVM01"  # string only
[datetime]$deadline = "2025-12-31"  # date/time
[bool]$isRunning = $true        # boolean only
```

`[int]$port = 443` → declares port as integer; attempting $port = "hello" will throw an error

`[datetime]$deadline = "2025-12-31"` → PowerShell converts the string to a DateTime object; you can now call date methods on it

`[bool]$isRunning = $true` → boolean accepts only $true, $false, 1, or 0

DateTime Methods

```
[datetime]$d = "2024-09-19"
$d.AddDays(20)        # returns date 20 days later
$d.DayOfWeek          # returns the name of the weekday
$d.Month              # returns the month number
```

String Methods

```
$s = "Hello PowerShell"
$s.Contains("Power")   # returns True
$s.IndexOf("P")        # returns 6 (zero-based position)
$s.ToUpper()           # returns "HELLO POWERSHELL"
$s.Split(",")          # splits string by comma into an array
$s.Replace("Hello","Hi")  # returns "Hi PowerShell"
```

⚠ NOTE: Use Get-Member to discover all methods and properties of any variable. Pipe your variable to Get-Member: $s | Get-Member. This shows every method available, along with their parameter signatures.

Type Testing and Conversion

```
$x = "100"
$x -is [string]    # True
$x -is [int]       # False
[int]$y = $x       # PowerShell converts "100" to integer 100 automatically
$y -is [int]       # True
```

PowerShell attempts automatic type conversion whenever you assign a value to an explicitly typed variable. If the conversion is possible (e.g., the string "100" → integer 100), it succeeds silently. If it is not possible (e.g., the string "hello" → integer), an error is thrown.

2.4 — The Pipeline and Get-Member

The pipeline | is the backbone of PowerShell scripting. It passes the output objects of one command directly as input objects to the next. The key insight is that what flows through the pipe is not text — it is structured objects with named properties. Every time you use a new command and are unsure what properties the objects have, pipe the output to Get-Member.

```
Get-Service | Get-Member
Get-Process | Get-Member
Get-ChildItem | Get-Member
```

Get-Member displays the TypeName of the object, all its Properties (data you can read), and all its Methods (actions you can call). This is your primary tool for exploring PowerShell objects.

⚠ NOTE: After running Format-Table, the objects in the pipeline change type from the original (e.g., ServiceController) to a Microsoft.PowerShell.Commands.Internal.Format type. This means you CANNOT sort or filter after Format-Table. Always sort and filter BEFORE formatting. Use Select-Object to narrow columns while preserving the original type.

2.5 — Where-Object (Filtering)

Where-Object filters the objects flowing through the pipeline, passing only those that match a condition. Think of it as a bouncer at a club — every object must satisfy the condition to pass through. Objects that do not meet the condition are discarded.

```
# Modern simplified syntax (PowerShell 3.0+)
```

```
Get-Service | Where-Object Status -eq "Running"
Get-Service | Where-Object Name -like "*sql*"

# Classic script block syntax
Get-Service | Where-Object { $_.Status -eq "Running" }
Get-Process | Where-Object { $_.WorkingSet -gt 100MB }

# Azure DevOps example: find large log files
Get-ChildItem "C:\logs" | Where-Object { $_.Length -gt 50MB }
```

`$_` → the current object being evaluated in the pipeline (also written as $PSItem)

`-eq` → equal to

`-ne` → not equal to

`-lt` → less than

`-gt` → greater than

`-le` → less than or equal to

`-ge` → greater than or equal to

`-like "*sql*"` → wildcard match — * matches any number of characters

⚠ NOTE: PowerShell comparison operators are text-based (not symbols). -eq not ==, -lt not <. This avoids ambiguity with XML and redirection operators. The -like operator supports wildcards: * for any sequence of characters, ? for exactly one character.

PHASE 3 — Syntax and Configuration

3.1 — Sort-Object

Sort-Object sorts the objects coming through the pipeline by one or more of their properties. It is one of the most frequently used pipeline commands because raw command output is rarely in a useful order. The sort is ascending by default; add -Descending to reverse it.

```
# Sort services alphabetically by name
Get-Service | Sort-Object Name

# Sort processes by memory use, largest first
Get-Process | Sort-Object VM -Descending

# Sort by multiple properties
Get-Service | Sort-Object Status, Name

# Alias: sort
Get-Process | sort VM -Descending
```

`Sort-Object Name` → sorts ascending by the Name property

`-Descending` → reverses the sort order

`Sort-Object Status, Name` → sorts by Status first; when Status values are equal, sorts by Name

⚠ NOTE: Service Status values are stored internally as numbers (Stopped=1, Starting=2, Running=4), not as text. Sorting by Status will show Stopped first (lowest number), not alphabetically. This often surprises people.

3.2 — Measure-Object

Measure-Object performs calculations on objects flowing through the pipeline. By default it counts them. Add switches to calculate sum, minimum, maximum, and average. When calculating numeric properties of objects (like file sizes), specify which property to use with -Property.

```
# Count objects
```

```
1, 2, 3, 4, 5 | Measure-Object

# Range notation shorthand
1..5 | Measure-Object -Sum -Minimum -Maximum

# Calculate total file size in a directory
Get-ChildItem "C:\Scripts" | Measure-Object -Property Length -Sum

# Count running services
Get-Service | Where-Object Status -eq "Running" | Measure-Object
```

`1..5` → range notation — shorthand for the list 1, 2, 3, 4, 5

`-Property Length` → tells Measure-Object which property to do the maths on

`-Sum -Minimum -Maximum` → multiple switches can be combined on one command

⚠ NOTE: File size is stored in the property named Length (not Size or FileSize). This is because Get-ChildItem returns not only files but also registry keys, certificates, and other objects — Length is the generic property name across all of them.

3.3 — Select-Object

Select-Object does two things: it limits the number of objects in the pipeline (First, Last, Skip), and it limits which properties are displayed. Think of it as both a row filter and a column filter for your data. Importantly, Select-Object preserves the object type — unlike Format-Table which destroys it.

```
# Show only the 3 biggest files
Get-ChildItem | Sort-Object Length -Descending | Select-Object -First 3

# Skip the largest file, show 2nd through 4th
Get-ChildItem | Sort-Object Length -Descending | Select-Object -Skip 1 -First 3

# Show only specific properties
Get-ChildItem | Select-Object Name, Length, LastWriteTime

# Show all properties
Get-ChildItem | Select-Object *

# Alias: select
Get-Service | select Name, Status
```

`-First 3` → keep only the first 3 objects

`-Last 3` → keep only the last 3 objects

`-Skip 1` → skip the first 1 object, then pass the rest

`Select-Object Name, Length` → keep only the Name and Length properties on each object

`Select-Object *` → show all available properties on each object

3.4 — Calculated Properties

A calculated property is a hash table you pass to Select-Object (or Format-Table) that defines a new computed column. You give it a name and an expression. The expression runs once per object and can use any math or string operation. The current object in scope is referenced as $PSItem or $_.

```
# Show file sizes in kilobytes instead of bytes
Get-ChildItem | Select-Object Name, @{n="SizeKB"; e={$PSItem.Length / 1KB}}

# Add a megabytes column too
Get-ChildItem | Select-Object Name,
  @{n="SizeKB"; e={"$("{0:N2}" -f ($PSItem.Length / 1KB))"}},
  @{n="SizeMB"; e={"$("{0:N2}" -f ($PSItem.Length / 1MB))"}}
```

`@{n="SizeKB"; e={...}}` → hash table defining calculated property: n = column name, e = expression
`n=` → short for "name" — also accepts "label" or "l" (avoid "l" as it looks like 1)
`e=` → short for "expression" — the code block that computes the value
`$PSItem.Length` → access the Length property of the current object ($_ is the older alias)
`1KB` → PowerShell constant = 1024 bytes. Also: 1MB = 1024KB, 1GB = 1024MB
`"{0:N2}" -f value` → format string: 0 = first value, N = numeric format, 2 = two decimal places

⚠ NOTE: $PSItem is the modern name (PowerShell 3.0+). The older $_ notation still works and is used heavily in existing scripts. They are identical in behaviour. In interviews, knowing both shows depth.

3.5 — ForEach-Object

ForEach-Object runs a script block once for every object that arrives from the pipeline. It is the pipeline-compatible equivalent of a foreach loop. Inside the block, the current object is available as $_ or $PSItem. ForEach-Object also supports Begin, Process, and End blocks for setup, per-object work, and teardown.

```
# Basic usage — write a message per service
Get-Service | ForEach-Object { Write-Host "Service: $($_.Name)" }

# Copy text files to a backup folder
Get-ChildItem | Where-Object { $_.Extension -eq ".txt" } |
  ForEach-Object { $_.CopyTo("C:\Backup\" + $_.Name) }

# Advanced: Begin / Process / End blocks
Get-Process | ForEach-Object `
  -Begin    { Get-Date | Out-File "C:\log.txt" } `
  -Process  { $_ | Select-Object Name, VM | Out-File "C:\log.txt" -Append } `
  -End      { Write-Host "Done" }
```

`-Begin { }` → runs ONCE before the first object is processed — use for setup (log headers, init)
`-Process { }` → runs for EACH object — the main per-object work
`-End { }` → runs ONCE after the last object — use for cleanup or summary
`Out-File -Append` → -Append adds to the file without overwriting existing content

⚠ NOTE: ForEach-Object (the cmdlet) and foreach (the loop statement) look similar but differ in pipeline usage. ForEach-Object is designed for the pipeline and receives objects one at a time. The foreach loop statement iterates over a collection stored in a variable. In pipelines, always use ForEach-Object.

3.6 — Data Export and Import

PowerShell can convert objects to formats like CSV, HTML, and XML for saving to disk or sharing with other tools. The ConvertTo-* cmdlets convert objects into text in the specified format. The Export-* cmdlets do the conversion and save the file in one step. Import-* and ConvertFrom-* reverse the process.

```
# Discover all conversion commands
Get-Help -Verb ConvertTo
Get-Help -Verb ConvertFrom

# Export services to CSV
Get-Service | Export-Csv "C:\services.csv" -NoTypeInformation

# Import CSV back as objects
$importedServices = Import-Csv "C:\services.csv"

# Save services as HTML
Get-Service | ConvertTo-Html | Out-File "C:\services.html"
```

```
# Save plain text output
Get-Process | Out-File "C:\processes.txt"
Get-Process | Out-File "C:\processes.txt" -Append  # add to existing
```

`Export-Csv -NoTypeInformation` → the -NoTypeInformation switch removes the #TYPE header line from the CSV

`ConvertTo-Html | Out-File` → ConvertTo converts to text; you must pipe to Out-File to save it

`Export-Csv` → combines conversion AND file saving in one step

⚠ NOTE: When you import a CSV back with Import-Csv, all values come back as strings. Type information is lost. If you exported a service status as "Running", it comes back as the text "Running" — not the enum value. This matters if you try to filter on Status after importing.

PHASE 4 — Output Formatting

4.1 — The Format Command Family

PowerShell has a family of Format-* commands that control how output is displayed on screen. They all accept a -Property parameter to specify which columns to show. The critical rule is: Format-* commands MUST be last in the pipeline. After a Format-* command, the objects are converted into formatting instructions, not real data. Sorting and filtering after Format-* will fail.

Command	Alias	Output Style	Best Used For
Format-Wide	fw	Multi-column names only	Long lists where only one property matters
Format-List	fl	Per-object block, property per line	Objects with many properties or long text values
Format-Table	ft	Tabular rows and columns	Comparing multiple objects across the same properties
Out-GridView	ogv	Interactive GUI window	Ad-hoc exploration and filtering interactively
Out-File	none	Saves to text file	Logging results to disk
Out-Printer	none	Sends to printer	Physical printing of output

4.2 — Format-Wide

Format-Wide displays output in a compact multi-column layout, showing only one property per object. Think of it like switching Windows Explorer from Detailed View to Small Icons view. It is useful when you have a long list and only care about one identifying property — like names.

```
Get-Service | Format-Wide              # auto-decides property and columns
Get-Service | fw -Property DisplayName  # show DisplayName instead
Get-Service | fw -Column 4              # force 4 columns
Get-Service | fw -AutoSize              # let PowerShell compute best column count
```

⚠ NOTE: -AutoSize computes the best column count based on the longest value in the set. If one service has an extremely long name, PowerShell may decide only 1 column fits — the result can be surprising but is always correct.

4.3 — Format-List

Format-List displays each object as a block, with each property on its own line. This is ideal for objects with many properties or where property values are too long to fit in a table column. Piping to fl * shows every single property of each object — extremely useful for discovery.

```
Get-Service | Format-List              # default properties, list format
Get-Service | fl *                      # show ALL properties of each service
Get-Service | fl Name, Status           # show only Name and Status
Get-Service "wuauserv" | fl *           # inspect one service in detail
```

`fl *` → the asterisk means all properties — this is the fastest way to see everything an object holds

4.4 — Format-Table

Format-Table displays output in rows and columns, the most familiar table format. It is the default for many commands. It supports several powerful parameters: -AutoSize removes wasted space, -Wrap prevents text truncation by breaking long values across lines, -GroupBy separates rows into labelled groups.

```
Get-Service | ft Name, Status, DisplayName
Get-Service | ft -AutoSize              # tighten column widths
Get-Service | ft -Wrap                  # wrap long text instead of truncating

# GroupBy requires pre-sorting on the same property
Get-Service | Sort-Object Status | ft -GroupBy Status

# Calculated column inside Format-Table
Get-ChildItem | ft Name, @{n="SizeKB"; e={"{0:N2}" -f ($_.Length/1KB)};
formatString="N2"}

# Count by group (use Group-Object, not Format-Table)
Get-Service | Group-Object Status
```

`-GroupBy Status` → creates a visual group header for each Status value — requires prior sorting on Status

`Group-Object Status` → returns a count table: how many objects per Status value (different from Format-Table -GroupBy)

⚠ NOTE: Format-Table -GroupBy requires the data to be sorted on the same property first. If you pipe unsorted data, you get multiple groups for the same status value scattered throughout. Always: Sort-Object Status | ft -GroupBy Status.

4.5 — Out Commands and HTML Export

The Out-* family sends formatted output to different destinations. By default, every command you run in PowerShell has its output sent to Out-Host (the console screen) invisibly. You can redirect this to a file, a printer, a graphical grid, or suppress it entirely.

```
Get-Service | Out-File "C:\svc.txt"         # save to file
Get-Service | Out-GridView                   # interactive filterable grid window
Get-Service | Out-Printer                    # send to default printer
Get-Service | ConvertTo-Html | Out-File "C:\svc.html"  # save as HTML page
Get-Service | Out-Null                       # discard output silently
```

⚠ NOTE: Out-GridView opens an interactive graphical window where you can filter, sort, and search the data interactively. This is extremely useful for ad-hoc investigation but not available on Server Core installations (no GUI).

PHASE 5 — Scripting and Program Flow

5.1 — WMI: Get-WmiObject

WMI (Windows Management Instrumentation) is a Windows infrastructure for querying and managing operating system data. Get-WmiObject is the PowerShell cmdlet that queries WMI. You pass it a class name (like Win32_LogicalDisk) and optionally a filter. The filter uses WQL (WMI Query Language), which resembles SQL syntax with single-quoted string values.

```
# Get all logical disks
Get-WmiObject Win32_LogicalDisk

# Get only the C: drive (WQL filter — note single quotes inside)
Get-WmiObject Win32_LogicalDisk -Filter "DeviceID = 'C:'"

# Use a variable for the drive letter
$letter = "C:"
Get-WmiObject Win32_LogicalDisk -Filter "DeviceID = '$letter'"

# Get disk space info
Get-WmiObject Win32_LogicalDisk -Filter "DeviceID = 'C:'" |
  Select-Object DeviceID, Size, FreeSpace, @{n="UsedSpace"; e={$_.Size -
$_.FreeSpace}}
```

`-Filter "DeviceID = 'C:'"` → WQL filter — the drive letter must be in single quotes inside the filter string

`$_.Size - $_.FreeSpace` → calculated property: UsedSpace derived from Size minus FreeSpace

⚠ NOTE: WQL (WMI Query Language) requires string values in the filter to be enclosed in single quotes. When building the filter in a PowerShell double-quoted string, escape the single quotes with a backslash: 'C:'. This is a specific syntax requirement for WQL, not a PowerShell rule.

5.2 — IF Statement

The IF statement executes a block of code only when a condition is true. It is the fundamental decision-making construct. In a real script, you use IF to handle different environments, validate inputs, respond to system state, or branch logic based on what the script discovers at runtime.

```
# Basic if
if ($diskFree -lt 10GB) {
    Write-Host "WARNING: Low disk space"
}

# if / elseif / else
$ValueToReturn = "freespace"

if ($ValueToReturn -eq "freespace") {
    Get-WmiObject Win32_LogicalDisk -Filter "DeviceID = 'C:'" | Select-Object
FreeSpace
} elseif ($ValueToReturn -eq "totalspace") {
    Get-WmiObject Win32_LogicalDisk -Filter "DeviceID = 'C:'" | Select-Object Size
} else {
    # default: used space
    Get-WmiObject Win32_LogicalDisk -Filter "DeviceID = 'C:'" |
        Select-Object @{n="UsedSpace"; e={$_.Size - $_.FreeSpace}}
}
```

`if ($condition) { }` → runs the block only when condition is true

`elseif ($condition) { }` → checked only if all previous conditions were false

`else { }` → runs when ALL previous conditions were false; no condition needed

⚠ NOTE: Only the if block is mandatory. elseif and else are optional. You can have zero or many elseif blocks, but at most one else. The else block catches everything that did not match any earlier condition — it acts as a safe fallback or default.

Real-world uses for IF in DevOps scripts: test if a file exists before reading it, check free memory before running a heavy operation, verify the current user is an administrator before running privileged commands, check if a service is running before attempting to stop it.

5.3 — SWITCH Statement

The SWITCH statement is a cleaner alternative to a long chain of IF/ELSEIF blocks when you need to compare a single control variable against many known values. Each case block runs only when the control variable matches that case value. The default block runs when no case matches.

```
$ValueToReturn = "freespace"

switch ($ValueToReturn) {
    "freespace" {
        Get-WmiObject Win32_LogicalDisk -Filter "DeviceID = 'C:'" | Select-Object
FreeSpace
    }
    "totalspace" {
        Get-WmiObject Win32_LogicalDisk -Filter "DeviceID = 'C:'" | Select-Object
Size
    }
    default {
        Get-WmiObject Win32_LogicalDisk -Filter "DeviceID = 'C:'" |
            Select-Object @{n="UsedSpace"; e={$_.Size - $_.FreeSpace}}
    }
}
```

`switch ($variable) { }` → evaluates $variable and jumps to the matching case

`"freespace" { }` → runs this block when $variable equals "freespace"

`default { }` → runs when no case matched — equivalent to the final else in an if chain

Aspect	IF / ELSEIF	SWITCH
Best for	Range checks, complex conditions, different variables per check	Single variable vs many known fixed values
Readability with 5+ conditions	Gets hard to follow quickly	Stays clean and scannable
Supports any condition	Yes	No — only equality checks by default
Best practice	Use for complex logic	Use when you know all possible values

⚠ NOTE: Using SWITCH instead of nested IF statements is a programming best practice because it makes the code much easier to read and maintain. If someone else needs to add a new case later, they add one block to SWITCH rather than tracking down the right place in a chain of IFs.

5.4 — WHILE Loop

The WHILE loop executes a block of commands repeatedly for as long as a condition remains true. The condition is evaluated BEFORE each iteration. This means if the condition is false on the very first evaluation, the block never runs at all. This is the key risk with WHILE — you must ensure the initial state satisfies the condition, or the loop is silently skipped.

```
$i = 0
while ($i -lt 10) {
    $i++          # increment MUST happen inside the block
    Write-Host $i
}
```

```
# break statement — exits the loop early
$i = 0
while ($i -lt 100) {
    $i++
    if ($i % 2 -eq 0) { Write-Host "$i is even" } else { Write-Host "$i is odd" }
    if ($i -eq 10) {
        Write-Host "Reached 10, breaking loop"
        break
    }
}
```

`$i = 0` → initialise the control variable BEFORE the while loop

`while ($i -lt 10) { }` → checks the condition; runs the block only while $i is less than 10

`$i++` → increment — MUST be inside the loop or it runs forever (infinite loop)

`break` → immediately exits the loop regardless of the current condition state

`$i % 2` → modulo operator — returns the remainder of $i divided by 2; 0 means even

⚠ NOTE: The most common WHILE bug: forgetting to increment the control variable inside the loop, resulting in an infinite loop. Always verify your loop will terminate. In DevOps use cases: WHILE is perfect for "wait until service is Running", "retry until ping succeeds", or "keep monitoring until log file appears".

5.5 — FOR Loop

The FOR loop is the classic counted loop. It is ideal when you know exactly how many times the loop should run. The three-part header defines: the initialisation (set control variable), the condition (keep going while true), and the increment (what to do after each iteration).

```
# Basic for loop — runs exactly 10 times
for ($i = 0; $i -lt 10; $i++) {
    Write-Host $i
}

# Loop over a list using for (index-based)
$servers = "AzVM01", "AzVM02", "AzVM03"
for ($i = 0; $i -lt $servers.Count; $i++) {
    Write-Host "Server: $($servers[$i])"
}
```

`for ($i = 0; $i -lt 10; $i++)` → $i = 0 initialises; $i -lt 10 is the condition; $i++ increments after each run

`$i++` → shorthand for $i = $i + 1

⚠ NOTE: When iterating over a list, prefer foreach over for. The foreach syntax is cleaner, eliminates the risk of off-by-one errors, and does not require you to manage the index. Use for only when you genuinely need the index value, or when you need to count a specific number of repetitions rather than enumerate a collection.

5.6 — FOREACH Loop and ForEach-Object

The foreach loop iterates over every element in a collection without requiring you to manage an index or counter. You declare an iteration variable that takes the value of each element in turn. This is simpler, safer, and more readable than the for loop when working with collections.

```
# foreach loop statement (works on in-memory collection)
$servers = "AzVM01", "AzVM02", "AzVM03"
foreach ($server in $servers) {
    Write-Host "Checking: $server"
    # could connect, run commands, copy files, etc.
}
```

```
# foreach-object cmdlet (works in pipeline)
$servers | ForEach-Object {
    Write-Host "Checking: $_"
}

# Pipeline example: copy file to each server
Get-ChildItem "C:\deploy" | Where-Object { $_.Extension -eq ".ps1" } |
    ForEach-Object { Copy-Item $_.FullName "\\$server\c$\scripts\" }
```

Aspect	foreach ($x in $list)	ForEach-Object (pipeline)
Where it runs	On an in-memory variable	In the pipeline receiving objects
Current item variable	$x (whatever you named it)	$_ or $PSItem
Begin/End blocks	Not supported	Supported (-Begin, -Process, -End)
Memory	Loads entire collection	Processes one object at a time
Best use	You already have a list variable	Chaining with other pipeline commands

PHASE 6 — Interview Favourites and Advanced Topics

6.1 — Aliases

An alias is a shortcut name for a cmdlet. PowerShell ships with dozens of pre-defined aliases, many designed to feel familiar to CMD and Unix users. ls and dir both work in PowerShell because they are aliases for Get-ChildItem. You can define your own aliases with Set-Alias, but they are lost when the session ends unless you save them to a profile script.

```
# List all aliases
Get-Alias

# Look up what an alias points to
Get-Alias ls          # → Get-ChildItem
Get-Alias ft          # → Format-Table

# Create a new alias
Set-Alias ShowFiles Get-ChildItem

# Delete an alias
Remove-Alias ShowFiles
# or via the drive:
Remove-Item Alias:\ShowFiles
```

`Get-Alias ls` → shows the definition: ls is an alias for Get-ChildItem

`Set-Alias ShowFiles Get-ChildItem` → creates alias ShowFiles pointing to Get-ChildItem

`Remove-Item Alias:\ShowFiles` → deletes the alias via the Alias: virtual drive

⚠ NOTE: Session-defined aliases are LOST when the PowerShell session ends. To make aliases permanent, add the Set-Alias commands to your PowerShell profile script ($PROFILE). The profile runs automatically every time a new session starts. Using obscure aliases in shared scripts is bad practice because others cannot read the code without looking up each alias.

6.2 — Getting Help: Get-Help, Update-Help, Get-Command

PowerShell ships without help files by default. You must download them once with Update-Help. After that, Get-Help is your primary reference. It shows the command syntax, all parameters, and — when online help is downloaded — full examples.

```
# Download/update help files (requires internet connection)
Update-Help

# Get help for a specific command
Get-Help Get-Service
Get-Help Get-Service -Detailed      # includes parameter descriptions
Get-Help Get-Service -Examples      # shows only practical examples
Get-Help Get-Service -Online        # opens the Microsoft docs page

# Discover commands by noun or verb
Get-Command -Noun Service           # all cmdlets that act on services
Get-Command -Verb Get               # all Get-* commands
Get-Command -Noun Variable          # all variable-related cmdlets

# Show-Command: GUI helper for building commands
Show-Command                        # lists all commands with search
Show-Command Get-Service            # opens form for Get-Service parameters
```

`Update-Help` → downloads help files for all installed modules from the internet

`Get-Help -Examples` → shows only the examples section — fastest way to see practical usage

`Get-Command -Noun Service` → discovers all cmdlets whose noun is "Service" — great for exploration

⚠ NOTE: Show-Command is underrated in interviews. It opens a GUI window where you can search for a command and fill in parameters visually, then click Run. The resulting command is also pasted back into the console so you can learn the syntax from what you just filled in.

6.3 — Start-Transcript and Stop-Transcript

Start-Transcript records everything that happens in a PowerShell session — every command entered and every result output — into a text file. This is invaluable for audit trails, troubleshooting, and documenting a session step by step. Stop-Transcript ends the recording.

```
Start-Transcript -Path "C:\logs\session_$(Get-Date -Format yyyyMMdd_HHmmss).txt"

# ... do your work here ...

Get-Date
Get-Service

Stop-Transcript

# Review the transcript
Get-Content "C:\logs\session_*.txt"
```

⚠ NOTE: Start-Transcript does NOT work in PowerShell ISE version 3.0. It works in the text console and in ISE from version 4 onwards. Whether a command works may depend on the host environment (console, ISE, VS Code, remote session) — this is a real gotcha that trips people up.

6.4 — Set-PSDebug -Strict

By default, PowerShell silently returns an empty value when you reference a variable that has not been declared. This makes typos in variable names very hard to catch. Set-PSDebug -Strict changes this behaviour — PowerShell will now throw an error if you reference a variable before it has been initialised. This is a defensive scripting practice.

```
Set-PSDebug -Strict

# Now this will throw an error if $ServerNam was never declared:
Write-Host $ServerNam    # typo! should be $ServerName
```

```
# Turn strict mode off
Set-PSDebug -Off
```

⚠ NOTE: Set-PSDebug -Strict is especially useful during script development. Always enable it when testing a new script so typos in variable names surface immediately. Remove it or use Set-StrictMode -Version Latest in production scripts for a more granular approach.

6.5 — The Alias: and Variable: Virtual Drives

PowerShell exposes certain internal namespaces as virtual drives that you navigate just like a file system. The most useful are Variable: (all session variables) and Alias: (all aliases). You use Get-ChildItem, Get-Item, and Remove-Item on these drives.

Virtual Drive	Contents	Example Command
Variable:	All session variables	Get-ChildItem Variable:
Alias:	All defined aliases	Get-ChildItem Alias:
Env:	Environment variables	Get-ChildItem Env:
Function:	All defined functions	Get-ChildItem Function:
HKLM:	HKEY_LOCAL_MACHINE registry	Get-ChildItem HKLM:\Software
HKCU:	HKEY_CURRENT_USER registry	Get-ChildItem HKCU:\Software

6.6 — Key Comparison: Select-Object vs Format-Table

This is one of the most interview-tested distinctions in PowerShell. Both commands narrow the properties you see, but they do fundamentally different things. Select-Object keeps the objects as objects (just with fewer properties). Format-Table converts them into formatting instructions for display. After Format-Table, you cannot filter or sort — the original type is gone.

Aspect	Select-Object	Format-Table
Output type	Same type as input (Selected.*)	Microsoft.PowerShell.Commands.Internal.Format
Can sort after?	Yes	No — type is destroyed
Can filter after?	Yes	No
Pipeline position	Anywhere in pipeline	Must be LAST in pipeline
Use for	Narrowing properties for further processing	Final display formatting only

⚠ NOTE: Remember: Format-* commands always come LAST. If you need to sort after narrowing columns, use Select-Object first, then Sort-Object, then Format-Table last. The rule: "format at the end, select in the middle".

6.7 — Summary: Comparison Operators Quick Reference

Operator	Meaning	Example
-eq	Equal to	$_.Status -eq "Running"
-ne	Not equal to	$_.Status -ne "Stopped"

-lt	Less than	$_.Length -lt 1MB
-gt	Greater than	$_.WorkingSet -gt 100MB
-le	Less than or equal	$_.Count -le 10
-ge	Greater than or equal	$_.FreeSpace -ge 5GB
-like	Wildcard match	$_.Name -like "*sql*"
-notlike	Wildcard non-match	$_.Name -notlike "*test*"
-match	Regex match	$_.Name -match "^az"
-is	Type check	$x -is [string]
%	Modulo (remainder)	$i % 2 -eq 0 (check if even)

— End of PowerShell Study Notes —